KLAi

Surprise, entropy and cross-entropy measure how well a model predicts what really happens

How does a language model turn raw scores into a probability for every next word, and how do we score it?

Mathematics for AI and Machine Learning | Session 17 of 18 | Probability and statistics
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Compute surprise, entropy, cross-entropy and KL divergence by hand, and show that cross-entropy = entropy + KL

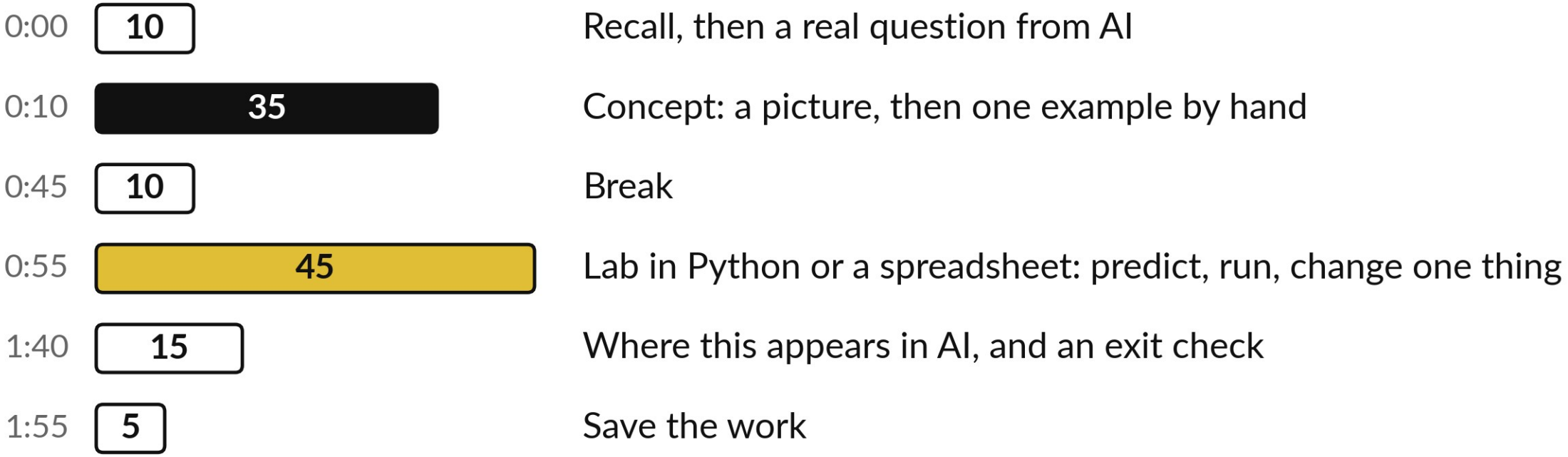


Reshape a softmax with temperature and top-p, and derive that the gradient of softmax with cross-entropy is prediction minus target (also for a sigmoid)



Compute the cross-entropy and perplexity of a toy language model on a six word sentence

Your two hours: information, sampling settings, then perplexity



Ideas first. Then the lab, in Python or a spreadsheet. Then AI.

Three questions from last time

1. What does maximum likelihood estimation choose?

2. Why do we work with the log-likelihood?

3. Which assumption about the target gives cross-entropy as the loss?

Answer aloud first. Then we check together.

Answers to the three questions from Session 16

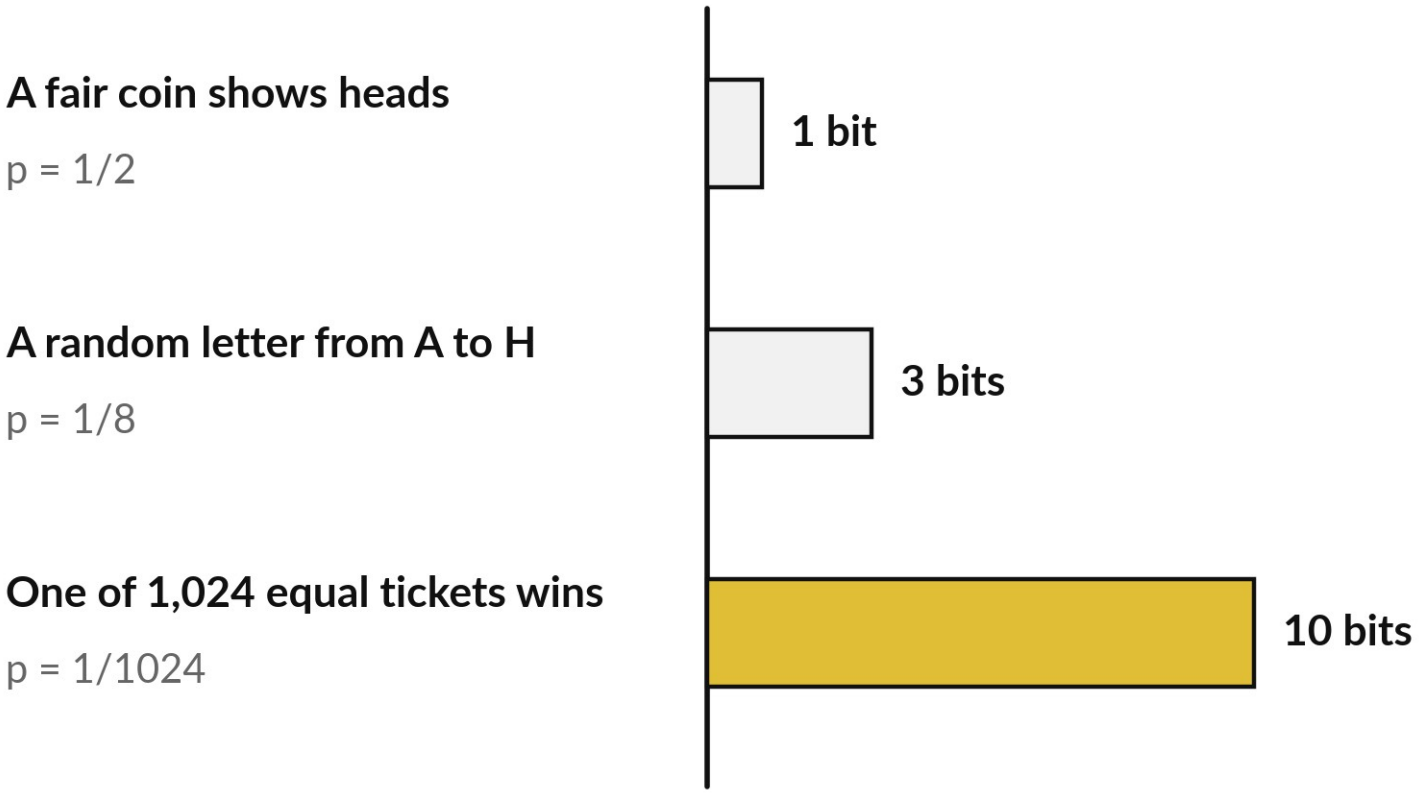
Question	Short answer
What does maximum likelihood estimation choose?	The parameter value that makes the observed data most probable, which means the largest likelihood.
Why do we work with the log-likelihood?	A product of many small probabilities underflows to zero. A sum of logarithms is stable, and the maximum stays at the same parameter.
Which assumption about the target gives cross-entropy as the loss?	A Bernoulli assumption for yes or no, or a categorical assumption for one of many classes. Gaussian noise gives squared error.

Today the Bernoulli and categorical losses get a second meaning, measured in bits as well as nats.

Words we use today

	Word	What it means
	**Entropy**	The average surprise of a distribution, in bits with log2 or nats with ln. How uncertain the outcome is.
	**Cross-entropy**	The average surprise when reality follows P but the model believes Q. The training loss for classification and language models.
	**KL divergence**	Kullback-Leibler divergence: cross-entropy minus entropy. The extra cost of believing Q instead of P.
	**Temperature and top-p**	Sampling settings. Temperature divides the logits before the softmax. Top-p keeps only the most likely words.
	**Perplexity**	e to the power of the mean negative log-probability. The effective number of equally likely choices the model has at each word.

Surprise: a rare event carries more information than a common one



The surprise (self-information) of an event with probability p is -log2(p) bits.

A bit is the answer to one yes or no question.

Every time the probability is cut in half, the surprise grows by one bit.

Surprise is the negative logarithm of the probability

Surprise in bits

$$I(x) = -\log_2 p(x)$$

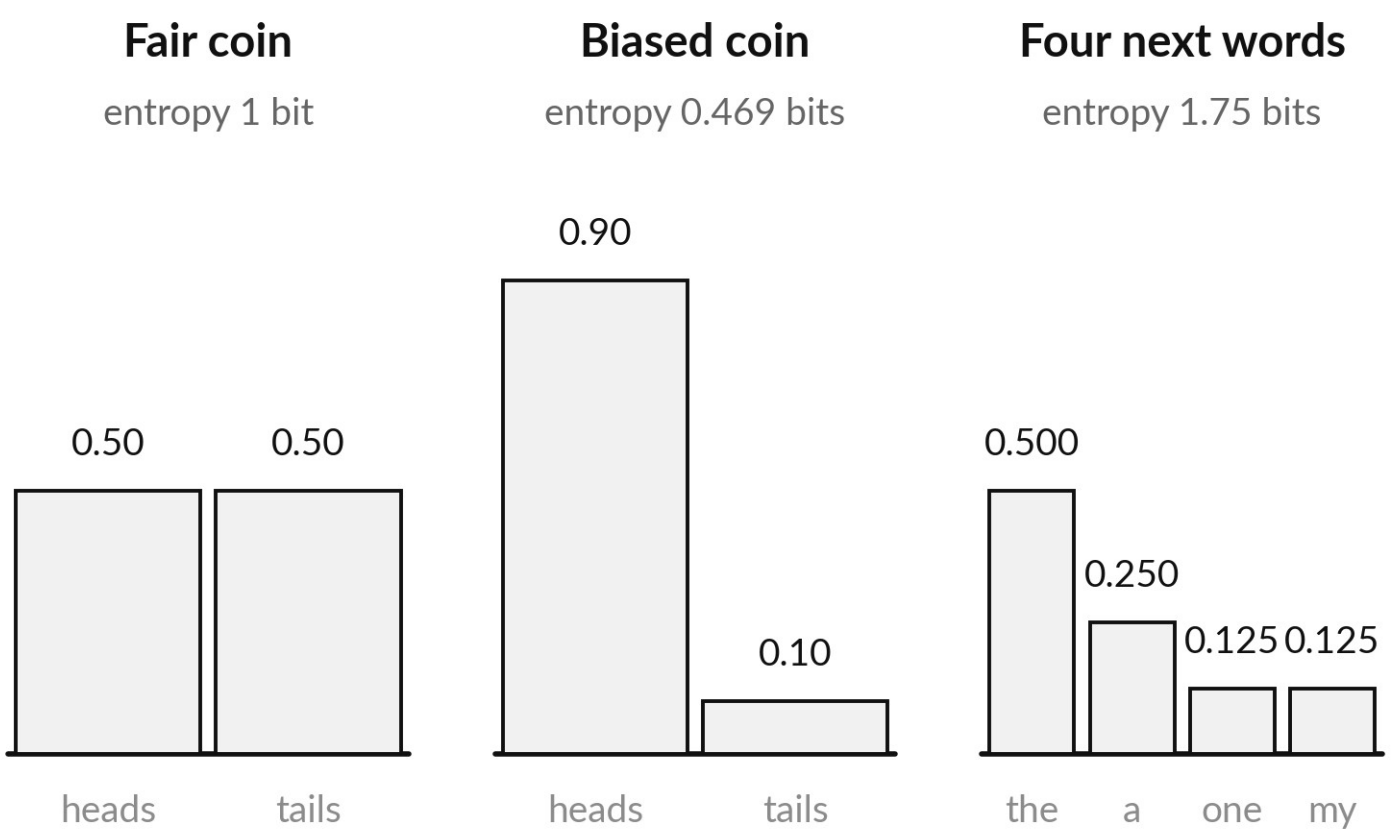
Surprise in nats

$$I(x) = -\ln p(x)$$

Symbol	What it means	In our data
x	One outcome, such as one word.	a random letter from A to H
p(x)	The probability of that outcome.	1/8
I(x)	The surprise (self-information) of that outcome.	3 bits = 2.079 nats
nat	The unit when we use ln. One bit is 0.693 nats.	libraries use ln

Bits = nats / 0.693. Session 16's cross-entropy of 0.299 nats is 0.431 bits.

Entropy is the average surprise, and it is largest when every outcome is equally likely



Entropy H(P) is the average surprise of outcomes drawn from the distribution P.

Among two-outcome distributions the fair coin is hardest to predict: 1 bit. The 0.9 coin has only 0.469 bits.

With K equally likely outcomes the entropy is log2 K: 2 bits for 4 words. Our uneven four words have 1.75 bits.

By hand: the entropy of four next words is the sum of p times surprise, 1.75 bits

Next word	p	Surprise (bits)	p × surprise
the	0.5	1	0.5
a	0.25	2	0.5
one	0.125	3	0.375
my	0.125	3	0.375
Total	1		1.75

Illustrative (made-up) probabilities. The total of the last column is the entropy.

Entropy is the probability-weighted average of the surprise

Entropy

$$H(P) = -\sum_i p_i \log_2 p_i$$

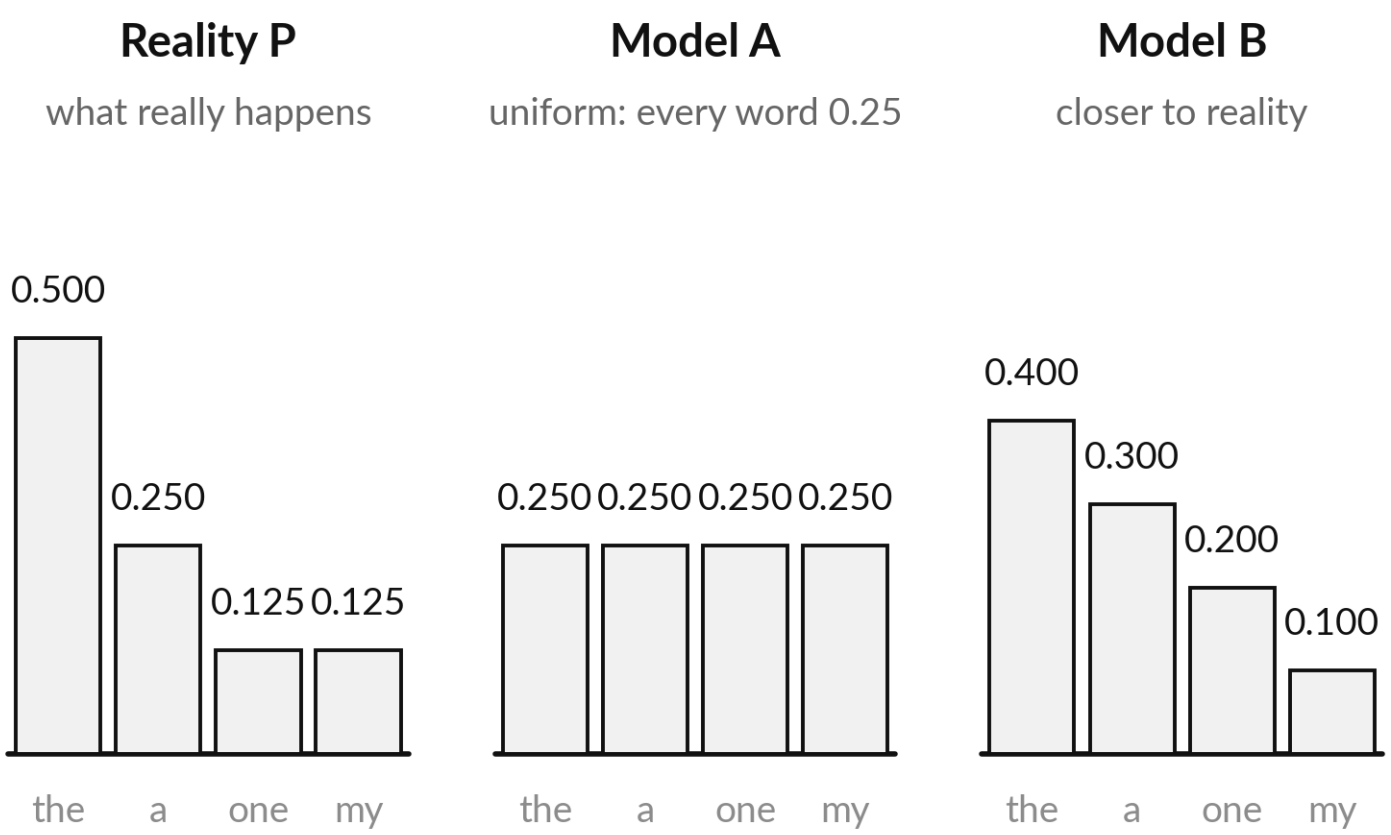
Our four words

$$H(P) = 0.5 + 0.5 + 0.375 + 0.375 = 1.75 \text{ bits}$$

Symbol	What it means	In our data
P	A distribution: probabilities that add to 1.	the four next words
p_i	The probability of outcome i.	0.5, 0.25, 0.125, 0.125
sum over i	Add the terms for every outcome.	four terms
H(P)	The entropy, in bits when the log is base 2.	1.75 bits

Four equally likely words would have the largest entropy: 2 bits.

Cross-entropy is the average surprise when reality follows P but the model believes Q



Cross-entropy H(P, Q) averages the model's surprise over what really happens.

Model A: H(P, Q) = 2.000 bits.

Model B: H(P, Q) = 1.801 bits. Smaller, because Q is closer to P.

Cross-entropy weights the model's surprise by the true probabilities

Cross-entropy

$$H(P, Q) = -\sum_i p_i \log_2 q_i$$

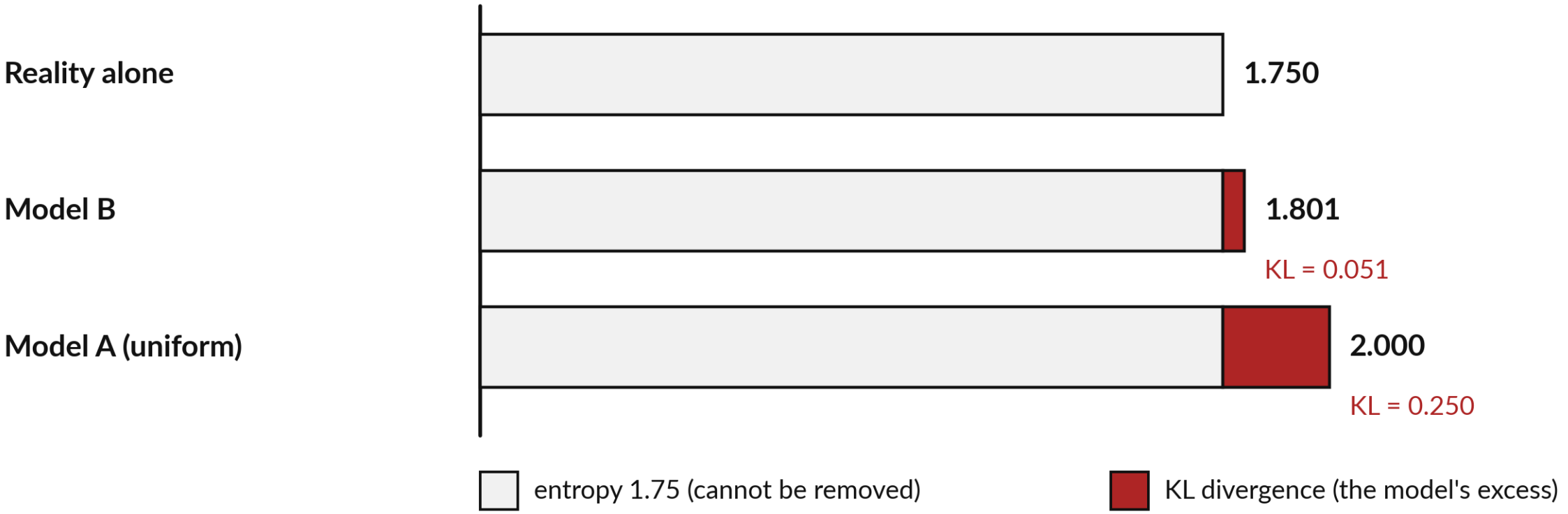
Model A

$$H(P, Q_A) = (0.5 + 0.25 + 0.125 + 0.125) \times 2 = 2.000$$

Symbol	What it means	In our data
p_i	The true probability of outcome i.	0.5, 0.25, 0.125, 0.125
q_i	The probability the model gives to outcome i.	A: 0.25 each
log2 q_i	The model's surprise at outcome i is minus this.	A: 2 bits for each
H(P, Q)	The cross-entropy, in bits.	A: 2.000, B: 1.801

With a real label, p is 1 on the true class and 0 elsewhere, so the loss is -log q of the true class.

Cross-entropy is entropy plus KL divergence: the entropy is fixed, the KL is what the model can improve



Model A: 1.750 + 0.250 = 2.000. Model B: 1.750 + 0.051 = 1.801. All in bits.

KL divergence is never negative and is zero only when the model equals reality

KL divergence

$$D_{KL}(P \parallel Q) = \sum_i p_i \log_2 \frac{p_i}{q_i}$$

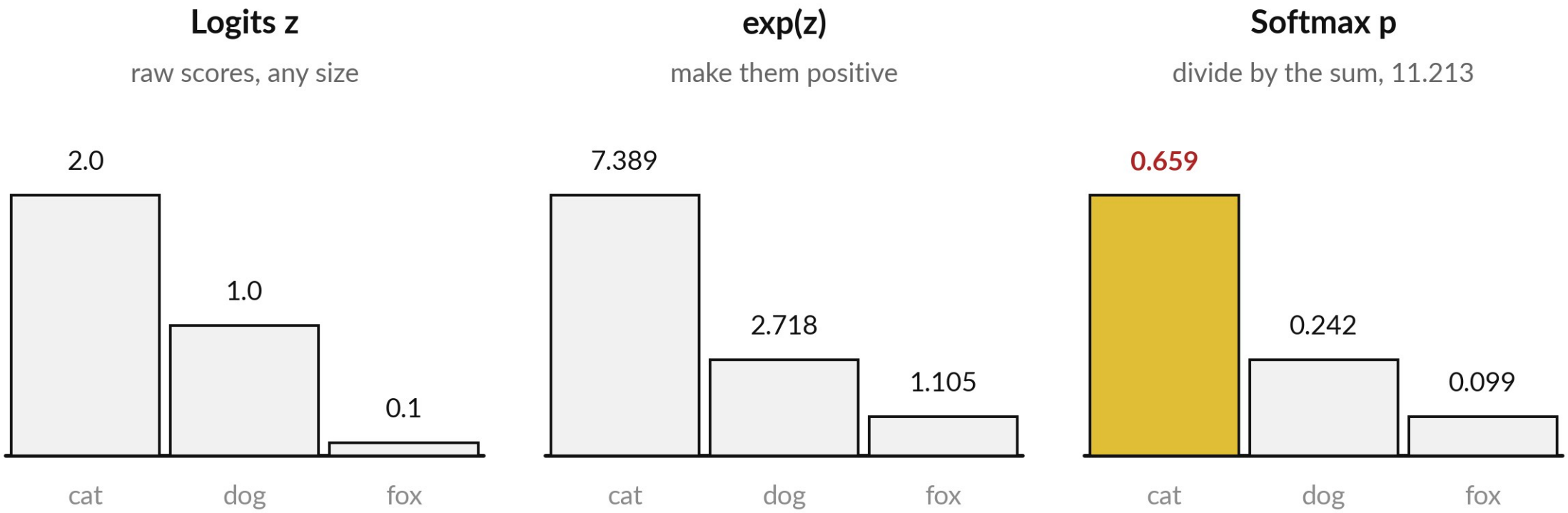
The link

$$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$$

Symbol	What it means	In our data
D_KL(P || Q)	The extra cost of believing Q when reality is P.	A: 0.250, B: 0.051 bits
Never negative	It is 0 only when Q = P everywhere.	a perfect model
Not symmetric	D(P || Q) is usually not D(Q || P).	so it is not a distance
H(P)	The entropy of reality.	1.75 bits

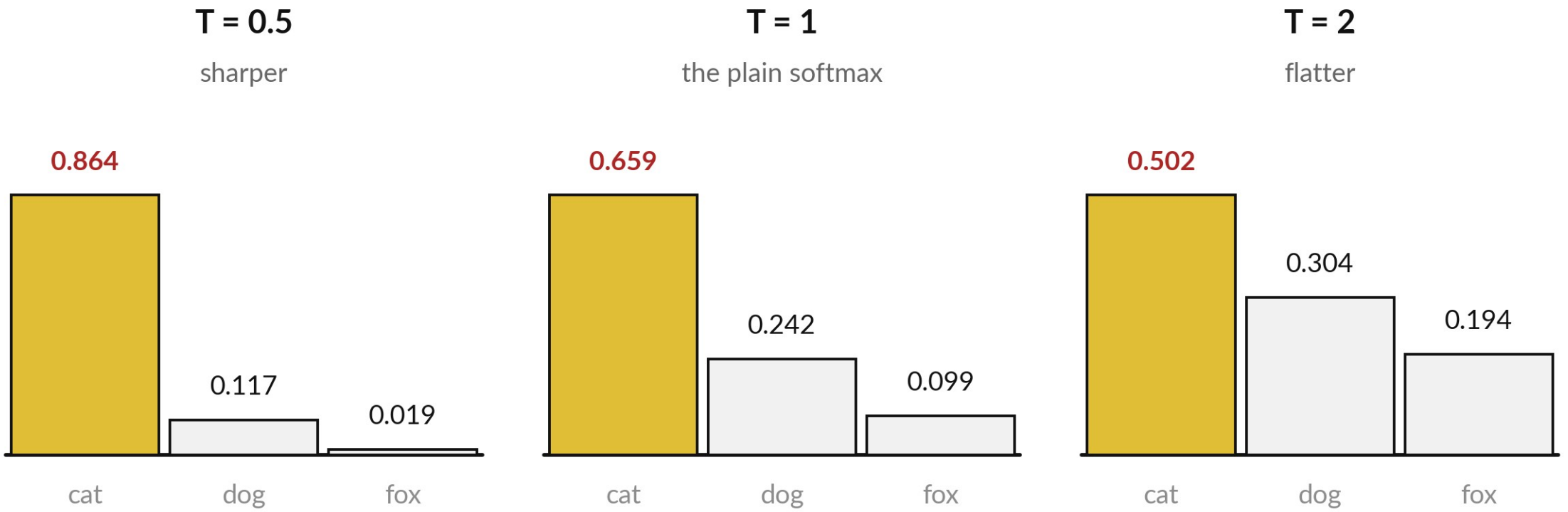
Minimising cross-entropy over the model is the same as minimising KL, because H(P) is fixed.

Recall from Session 12: the softmax turns logits into probabilities, and the stable form subtracts the largest logit



Illustrative logits. Stable form (Session 12): subtract the largest logit first. Same p.

Temperature divides the logits before the softmax: a low T sharpens, a high T flattens



Each panel adds to 1. T near 0: the top class takes all (greedy). Large T: equal shares.

Softmax with temperature: one division, then the same softmax

Softmax with temperature

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

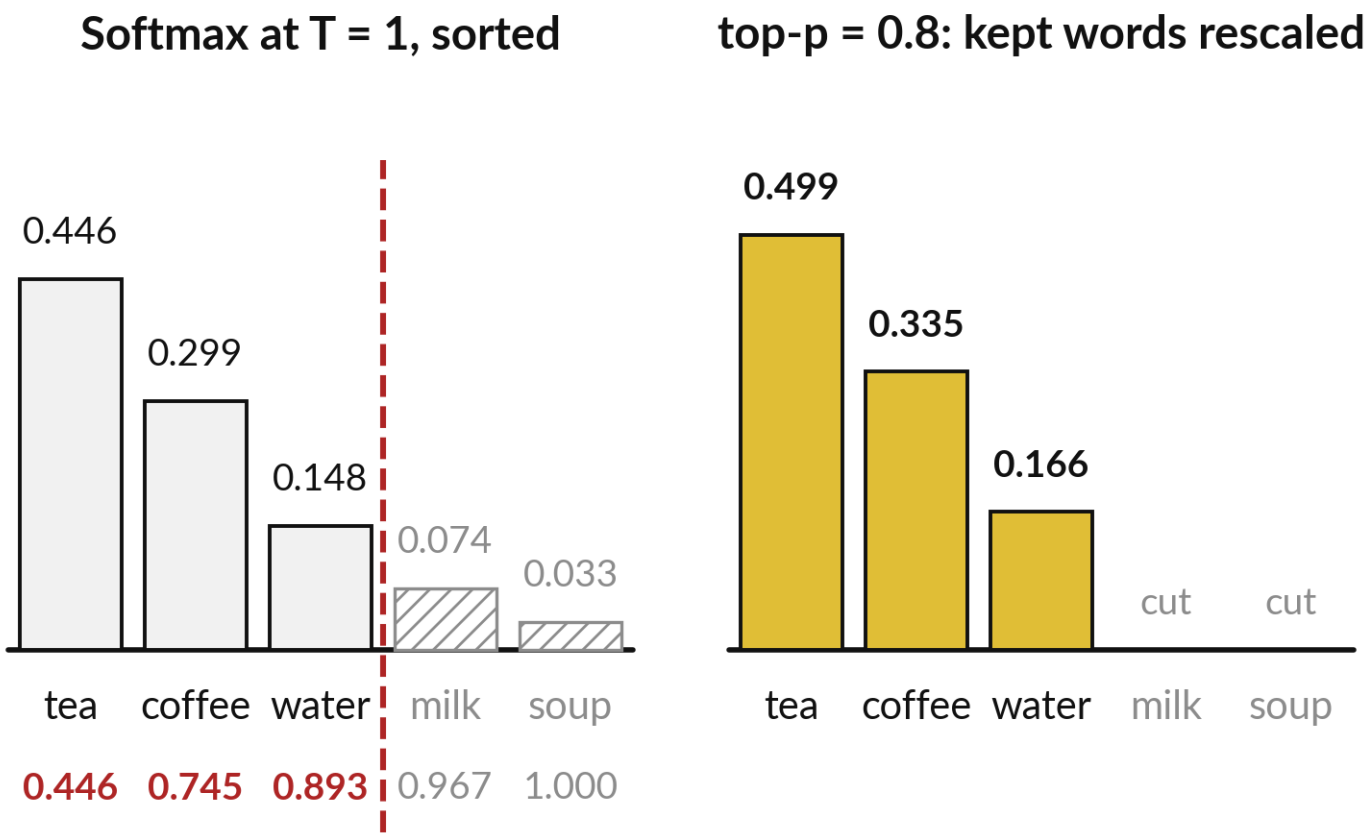
Cat at T = 2

$$\frac{e^1}{e^1 + e^{0.5} + e^{0.05}} = \frac{2.718}{5.418} = 0.502$$

Symbol	What it means	In our data
z_i	The logit (score) of class i.	2, 1, 0.1
T	The temperature, above 0. T = 1 is the plain softmax.	T = 2 halves every logit
p_i	The probability of class i at temperature T.	cat at T = 2: 0.502
Sum over j	All classes, the same T in every term.	3 terms

Use the stable form of Session 12 here too: subtract the largest z_j / T first.

Top-p (nucleus) sampling keeps the smallest set of likely words, then rescales them



Red: the running total. Cut once it reaches 0.8.

Top-p = 0.8: keep the most likely words until their running total reaches 0.8, then rescale them.

Tea, coffee and water reach 0.893. Milk and soup are cut. The draw of Session 14 then uses only the kept slices.

Divide by 0.893: tea 0.499, coffee 0.335, water 0.166.

Cross-entropy straight from the logits: log-softmax is log-sum-exp minus the true logit

Log-softmax

$$\ln p_k = z_k - \ln \sum_j e^{z_j}$$

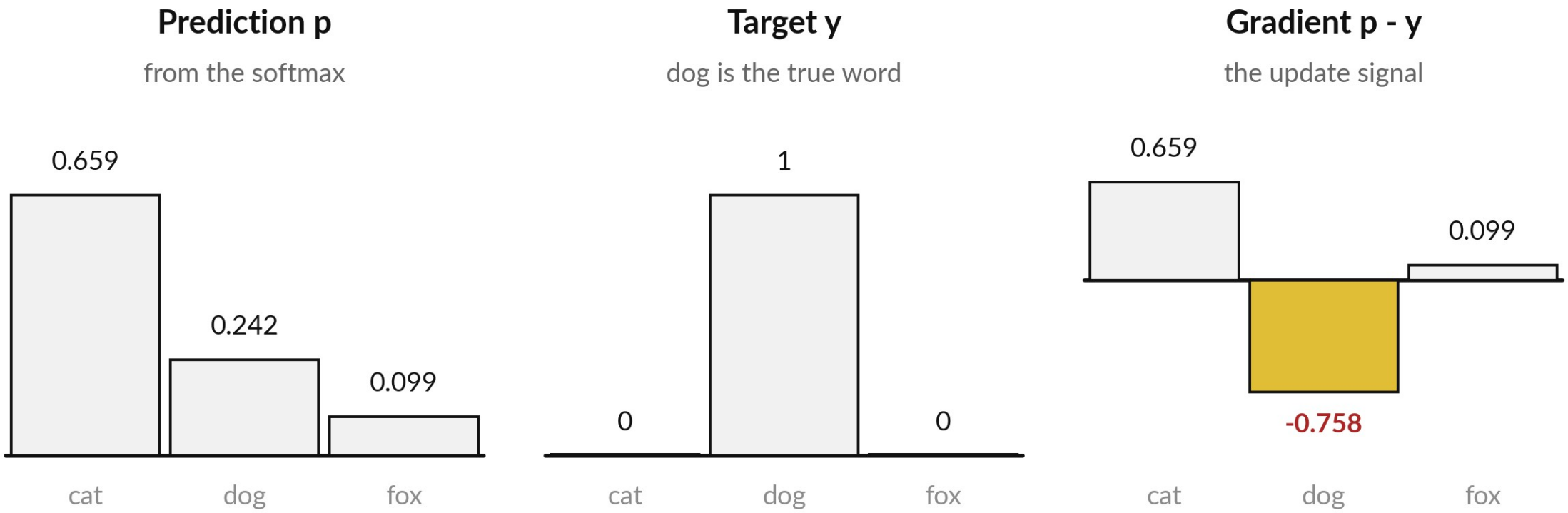
Loss, dog true

$$L = -\ln p_{dog} = \ln(11.213) - 1.0 = 2.417 - 1.0 = 1.417 \text{ nats}$$

Symbol	What it means	In our data
z_k	The logit of class k.	dog: 1.0
ln sum of e^z	The log-sum-exp of Session 12.	ln 11.213 = 2.417
L	The cross-entropy loss for one example, in nats.	1.417
ln bits	Divide by ln 2 = 0.693.	2.044 bits

Session 12 promised this link: libraries compute the loss this way, so no tiny probability is ever formed and logged.

Softmax with cross-entropy has a simple gradient with respect to the logits: prediction minus target



Illustrative. Dog, the true word, has a negative gradient: its logit should go up.

Derivation: the gradient of softmax with cross-entropy is p minus y

Slopes

$$\frac{\partial L}{\partial p_k} = -\frac{y_k}{p_k}, \qquad \frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j)$$

Chain rule

$$\frac{\partial L}{\partial z_j} = -\sum_k \frac{y_k}{p_k} p_k(\delta_{kj} - p_j) = -\sum_k y_k(\delta_{kj} - p_j) = p_j - y_j$$

Symbol	What it means	In our data
L	The loss: L = -sum of y_k ln p_k.	1.417 nats in our example
delta_kj	1 when k = j, else 0. Not the error signal of backpropagation.	picks one term
y_k	The one-hot target: 1 for the true class.	y = (0, 1, 0)
p_j - y_j	The gradient for logit j.	(0.659, -0.758, 0.099)

The 1 / p_k cancels p_k. The delta part gives y_j. The other part gives p_j, because the y_k add to 1.

By hand: how cat and dog move when the cat logit rises by 0.001

The middle slope of the derivation, p_k(delta_kj - p_j), with numbers. Softmax p = (0.659, 0.242, 0.099). Move only the cat logit, z1.

Same class	dp1/dz1 = p1(1 - p1) = 0.659 × 0.341 = 0.225. The quotient rule of Session 7 gives this.
Other class	dp2/dz1 = -p2 p1 = -0.242 × 0.659 = -0.160
Check	Recompute the softmax with z1 = 2.001: p1 rises by 0.000225 and p2 falls by 0.000160. Divide by 0.001: 0.225 and -0.160.

By hand: logits 2.0, 1.0, 0.1 with dog true give a loss of 1.417 nats

Illustrative three-class example. The numbers match the lab.

Softmax, target	p = (0.659, 0.242, 0.099) for cat, dog, fox and y = (0, 1, 0)
Loss	L = -ln 0.242 = 1.417 nats
Gradient	p - y = (0.659, -0.758, 0.099). The three entries add to 0.
Check	Add 0.001 to the cat logit: the loss rises by about 0.659 × 0.001 = 0.00066.

With two classes the softmax is a sigmoid, so sigmoid with binary cross-entropy also gives p minus y

Two classes

$$\frac{e^{z_1}}{e^{z_1}+e^{z_2}} = \frac{1}{1+e^{-(z_1-z_2)}} = \sigma(z_1 - z_2)$$

Gradient for one logit z

$$p = \sigma(z), \quad \frac{\partial L}{\partial z} = \left(-\frac{y}{p} + \frac{1-y}{1-p}\right)p(1-p) = p - y$$

Symbol	What it means	In our data
sigma	The sigmoid (Session 11). Slope p(1 - p).	sigma(0.5) = 0.622
z	One logit: the score for class 1.	z = 0.5
L	Binary cross-entropy (Session 16).	y = 1: L = -ln p
p - y	The gradient for z.	0.622 - 1 = -0.378

Session 18 uses this form in logistic regression and in the output of its small network.

Perplexity: e raised to the power of the mean negative log-probability of the real words

the next word	The	cat	sat	on	the	mat	total
probability the model gave it	0.2	0.1	0.25	0.5	0.4	0.05	
-ln(p)	1.61	2.30	1.39	0.69	0.92	3.00	9.90

Illustrative (made-up numbers): the probability a toy model gave to each real word of the sentence.

Mean -ln p = 9.90 / 6 = 1.651 nats. Perplexity = e^1.651 = 5.21.

Perplexity is the exponential of the cross-entropy per token

Perplexity

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \ln p(w_i \mid w_1, \ldots, w_{i-1})\right)$$

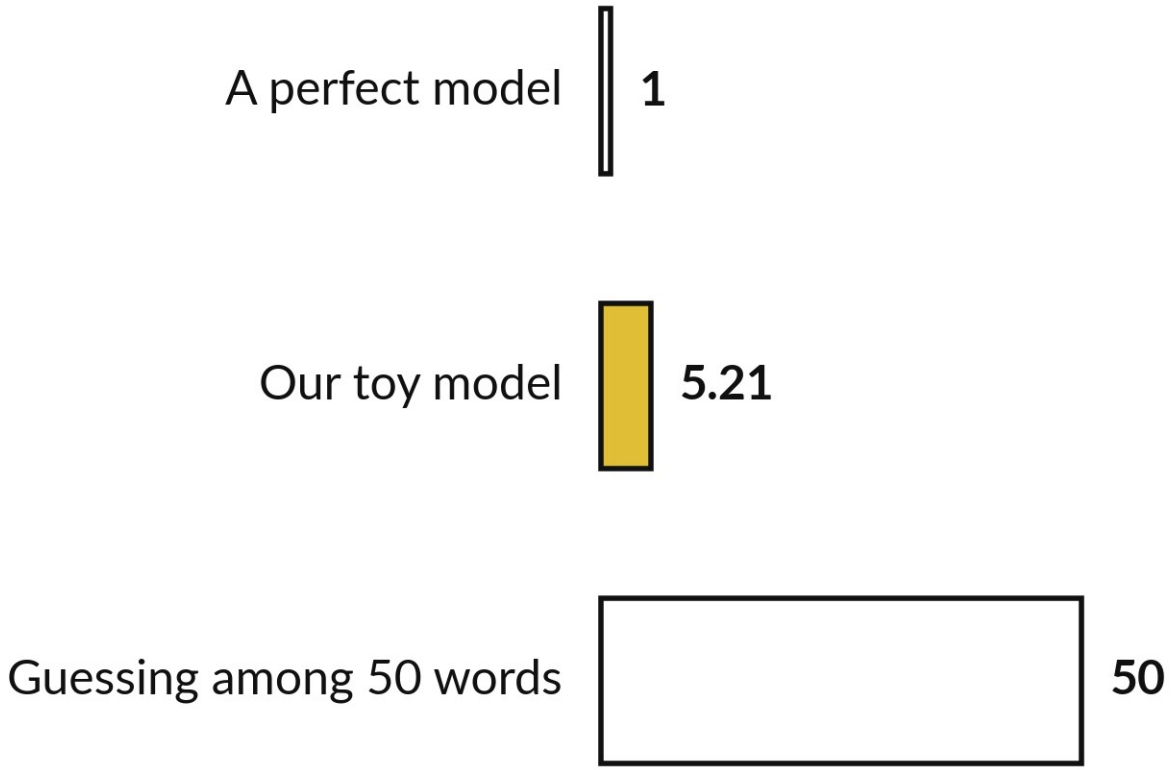
Our sentence

$$\mathrm{PPL} = e^{1.651} = 5.21$$

Symbol	What it means	In our data
N	The number of words (tokens) scored.	6
p(w_i | w_1 ... w_(i-1))	The probability the model gave to the real word i, given the words before it.	0.2, 0.1, 0.25, 0.5, 0.4, 0.05
Mean of -ln p	The cross-entropy per word, in nats.	1.651
PPL	Perplexity. In bits it is 2 to the power of the bits per word. Same value.	5.21

Lower is better. Perplexity 1 means the model gave probability 1 to every real word.

Perplexity reads as the effective number of equally likely words the model chooses between



A perplexity of 5.21 means the model is as unsure as if it picked among about five equally likely words.

Illustrative numbers.

Compare perplexities only for the same test text and the same tokens.

Three common mistakes about information measures

The mistake

Cross-entropy and KL divergence are the same thing.

Bits and nats give different perplexities.

A higher temperature makes the model smarter.

The fix

They differ by the entropy of the data.

Same value if the exponent matches the log base.

It only flattens the probabilities.

Always say which log base, which text and which sampling settings.

Read the recipe before you write it: entropy, KL, temperature, top-p, gradient, perplexity

```
H = -np.sum(P * np.log2(P))
kl = np.sum(P * np.log2(P / Q))
p = softmax(z / T)
keep = np.searchsorted(np.cumsum(p5), 0.8) + 1
grad = p - y
ppl = np.exp(np.mean(-np.log(token_probs)))
```

In plain words

Line 1: entropy in bits.

Line 2: KL divergence. Cross-entropy = H + kl.

Line 3: softmax with temperature T.

Line 4: how many words top-p keeps.

Line 5: the gradient for the logits.

Line 6: perplexity.

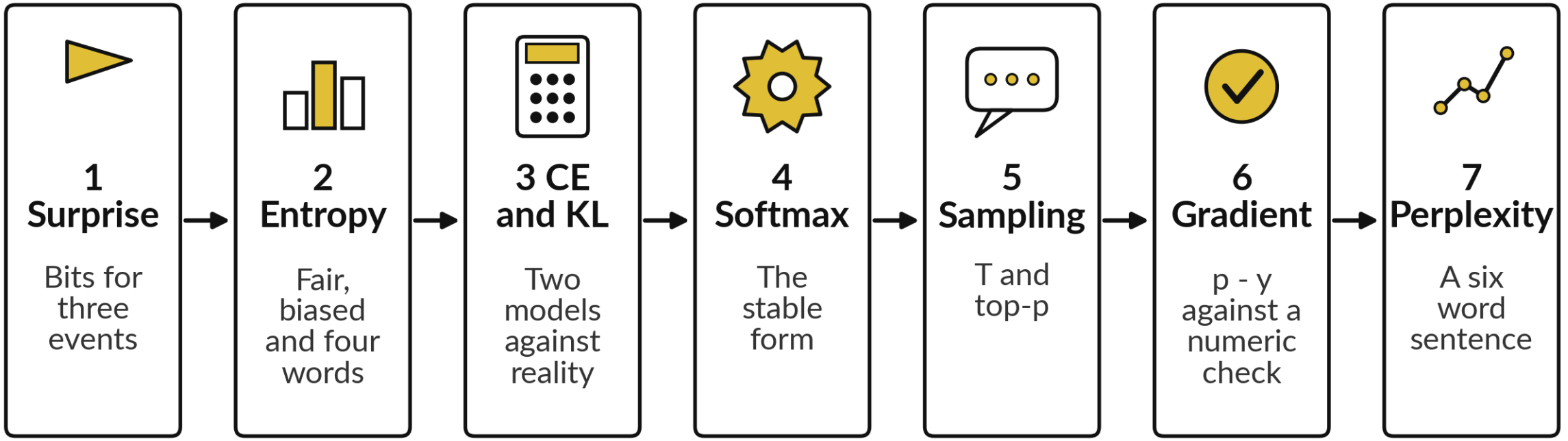
Spreadsheet: LOG(q, 2), SUMPRODUCT, a growing SUM, EXP.



Part 2: the lab

Close the slides. Open the notebook or the workbook.

The lab has seven steps, from surprise to perplexity



Python: Steps 1 to 7. Spreadsheet: sheets Entropy (Steps 1 to 3), Softmax (4 and 6), Sampling (5) and Perplexity (7). Predict, run, change one thing.

Check your numbers against these before you move on

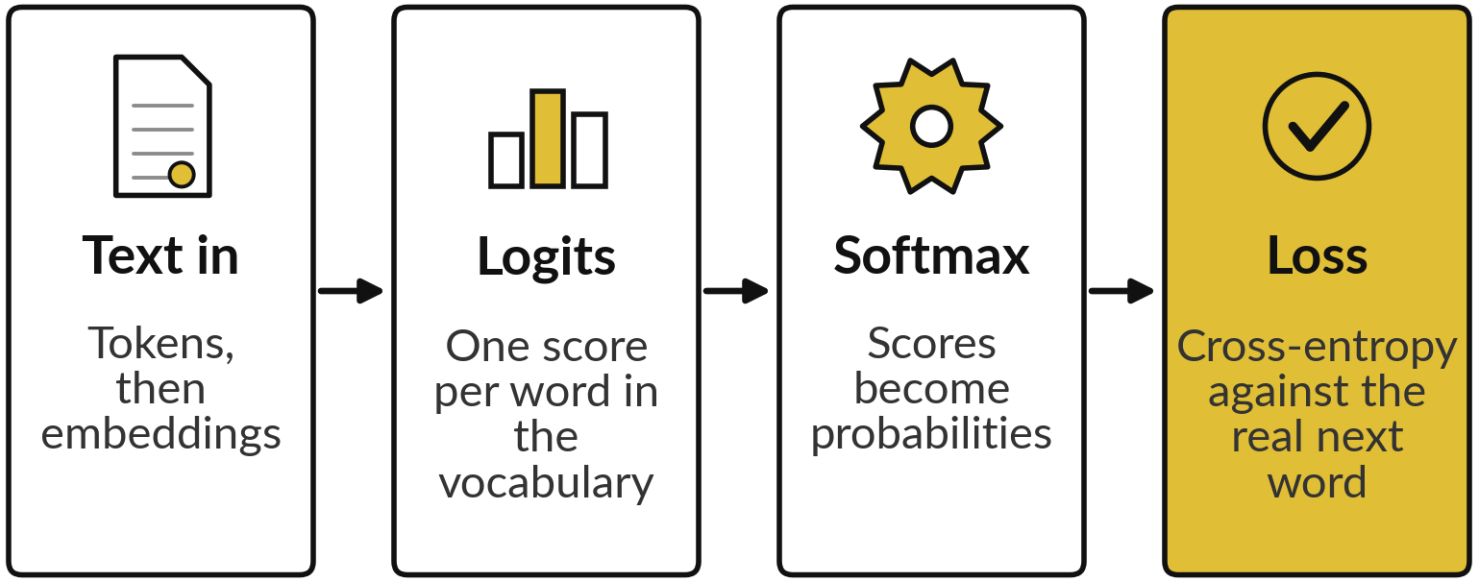
What you compute	You should see
Surprise of 1/2, 1/8, 1/1024 (bits)	1, 3, 10
Entropy of P, fair coin, 0.9 coin	1.75, 1, 0.469 bits
Cross-entropy and KL of model A and model B	2.000, 1.8007; 0.250, 0.0507 bits
Softmax of 2.0, 1.0, 0.1 (cat, dog, fox)	0.6590, 0.2424, 0.0986
Cat at T = 0.5 and 2; top-p keeps; tea rescaled	0.8638, 0.5017; 3 drinks; 0.4992
Loss and gradient p - y (dog true)	1.4170 nats; 0.6590, -0.7576, 0.0986
Sum of -ln p, mean, perplexity	9.903, 1.6506 nats, 5.210

If your number differs, stop and ask. Find the step that went wrong. Do not change a number to match.

Errors you will meet, what they mean and what to do

What you see	What it means	What to do
nan from the softmax	The exponential overflowed	Subtract the largest logit first
NOT YET: write your code in the cell above	An answer cell (marked gold) still says None	Replace None with your code and run both cells again
A negative KL divergence	P and Q are swapped, or they do not add to 1	Check both add to 1 and keep the order P then Q
Spreadsheet shows #DIV/0! or #NUM!	A probability of 0 inside a logarithm	Use only probabilities above 0

Where this appears in AI: pre-training by minimising cross-entropy on the next token



Published research: transformer language models are pre-trained by minimising next-token cross-entropy.

The p - y gradient from today is the signal that starts backpropagation.

Later stages, such as learning from human feedback, use other objectives. Closed products do not publish their internals.

Where this appears in AI: temperature and top-p are settings in public APIs

Sampling settings in public APIs (names only)

temperature

Documented by OpenAI, Gemini, Anthropic

top_p (nucleus)

Documented by the same three APIs

top_k

Documented by some providers

The next word is drawn at random, so answers can differ. Temperature and top-p set how much.

Low temperature: steadier answers. High: more varied.

Nucleus sampling: Holtzman and colleagues, 2020.

Products may add steps we cannot see.

Check an AI assistant's answer against your own numbers

5 minutes Optional, with any assistant

Ask an assistant for the softmax of the logits 2.0, 1.0, 0.1, the same at temperature 0.5, and the cross-entropy loss if the second class is true.

Compare: 0.659, 0.242, 0.099; 0.864 at T = 0.5; loss 1.417 nats.

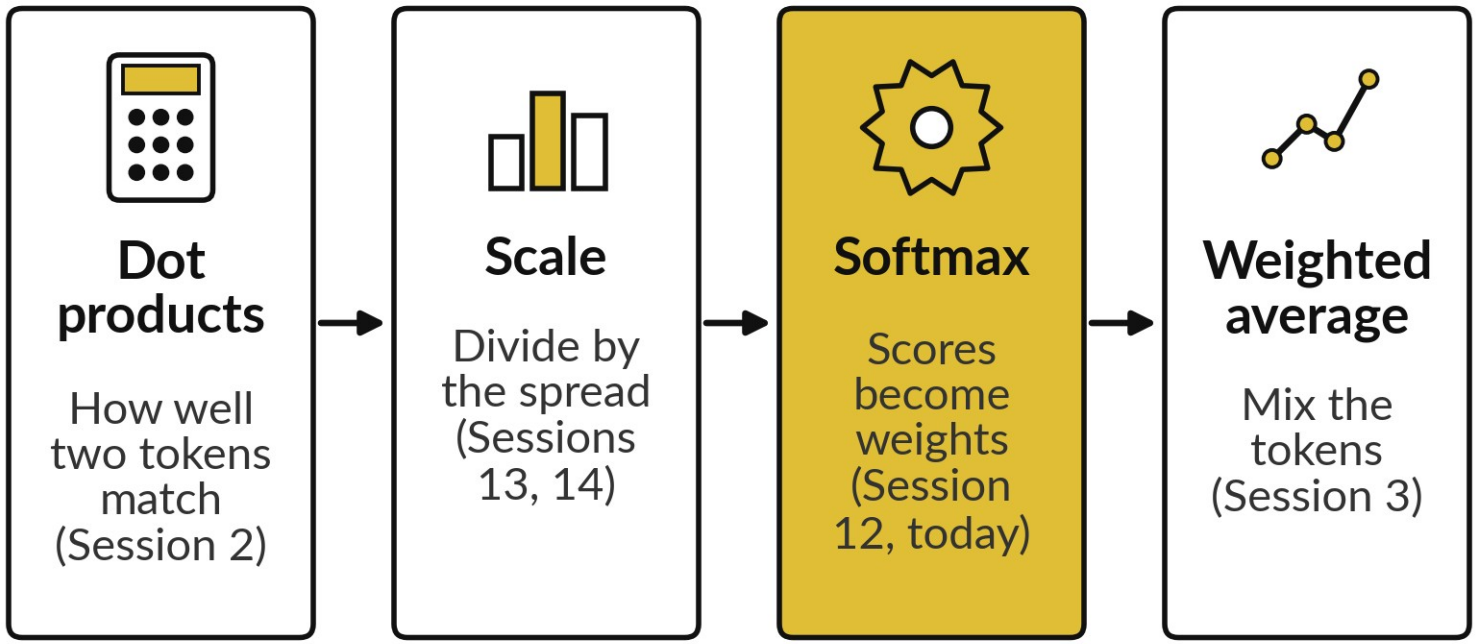
Ask which log base it used (bits: 2.04).

Never paste confidential data into an assistant.

You keep

A note: where it agreed with you, and where not.

Next session: attention puts today's softmax on top of dot products



Session 18 teaches attention, the core of a transformer, from parts you have.

Each token scores every other token, turns the scores into weights with a softmax, and takes a weighted average.

The new part is how the pieces fit together. You build it by hand.

Exit check: three questions and their short answers

Question	Short answer
What is the entropy of a fair coin, in bits?	1 bit.
What is the gradient of softmax with cross-entropy with respect to the logits?	The prediction minus the target: p - y. The same holds for a sigmoid with binary cross-entropy.
A model has a mean negative log-probability of 2.0 nats per word. What is its perplexity?	e^2 = 7.39.

Answer each in one sentence before you read the right-hand column.

Save your lab and keep six numbers

What you keep from today

Keep your notebook or workbook with all checks passing. Keep six numbers: entropy 1.75 bits, cross-entropy 2.000 bits, KL 0.250 bits, top class 0.864 at T = 0.5, gradient -0.758 and perplexity 5.21.

Our question today

How does a language model turn raw scores into a probability for every next word, and how do we score it?

Next session

Session 18: the course project. Build linear regression, logistic regression, PCA, a small network and attention from first principles.

Take away

Cross-entropy = entropy + KL
2.000 = 1.750 + 0.250 bits

Gradient
p - y = (0.659, -0.758, 0.099)

Toy perplexity
5.21

Next session
Course project: five builds